

Assessment 1: Analytical Problem Solving

Summary Report

1. Objective

This report summarises the approach, methodology, and results for the five analytical problem-solving exercises in Annexure A, covering classical search (water jug, 8-Queens, uniform cost search) and intelligent-agent design (Mars Rover, OLA Cab booking).

2. Methodology Summary

- **Water Jug Problem:** Modelled as a state-space search where each state is (amount in 4-gal jug, amount in 3-gal jug). Breadth-First Search (BFS) was used to guarantee the shortest sequence of fill/empty/pour actions reaching the goal state where the 4-gallon jug holds exactly 2 gallons.
- **Mars Rover Agent:** Analysed using the PEAS framework (Performance, Environment, Actuators, Sensors) to characterise percepts, environment properties, available actions, and a recommended agent architecture.
- **8-Queens Problem:** Formulated as a constraint-satisfaction problem and solved using backtracking search with column/diagonal conflict tracking, guaranteeing a valid non-attacking placement of all 8 queens.
- **OLA Cab Agent:** Modelled as a utility-based agent since the customer chooses among several satisfactory ride options based on trade-offs (fare, wait time, comfort) rather than a single fixed goal.
- **Uniform Cost Search:** Implemented with a priority queue (min-heap) that always expands the lowest cumulative-cost frontier node, guaranteeing the optimal least-cost path from the start warehouse S to the goal warehouse G.

3. Key Results

- Water Jug: 6 actions required to reach exactly 2 gallons in the 4-gallon jug.
- Mars Rover: best modelled as a model-based, utility-based agent.
- 8-Queens: a valid solution found in column order [0, 4, 7, 5, 2, 6, 1, 3].
- OLA Cab: utility-based agent design with pseudocode provided.
- UCS: optimal path S -> C -> D -> G with total cost 8 (using the demonstration graph in Python_Code.py).

4. Tools Used

Python 3 was used throughout, with the standard library (collections, heapq) for search algorithms, and matplotlib/networkx for graph visualisation of the UCS result (see Output.png).

5. Conclusion

All five problems were solved using appropriate search and agent-design techniques from classical AI: uninformed search (BFS, UCS), constraint-satisfaction/backtracking search, and the PEAS/agent-type framework for designing rational agents. Full working code is provided in Python_Code.py, and detailed step-by-step solutions are in Solution.pdf.